

Análisis de vulnerabilidades — CiberDojo

📅 21 sept 2026 · @Raul Chavez

Análisis estático de código (sin pruebas de intrusión contra el sistema en producción) sobre el commit `41569fc` del repositorio `shield-ecuador-app`. La mayoría de los hallazgos de este documento corresponden a un ciclo de auditoría y corrección ya ejecutado fila por fila (ver `DESIGN_REVIEW_CHECKLIST.md`); aquí se listan de forma independiente, con su estado actual verificado.

Resumen ejecutivo

Se identificaron **6 hallazgos activos**: 0 críticos, 0 altos, 3 medios y 3 bajos. Ninguno permite ejecución de código remoto, fuga de credenciales ni acceso no autorizado a datos de otro usuario en el estado actual del código.

Esto es el resultado de un ciclo de auditoría ya cerrado: antes de esta revisión existían hallazgos de severidad alta (XSS por HTML sin escapar, SSRF en webhooks y fuentes de noticias, inyección de fórmulas en CSV, acceso anónimo a funciones de pago, aprobación de análisis sin auditar, entre otros); todos fueron corregidos y verificados con pruebas automatizadas antes de este documento. Lo que queda abajo es lo que siguió sin resolver tras ese ciclo.

Tabla de hallazgos

Severidad	Componente	Hallazgo	Estado	Evidencia
Media	<code>frontend/</code> (dependencias de desarrollo)	29 avisos de <code>npm audit</code> (14 altos) en <code>react-scripts</code> y su cadena de build (jest,	Pendiente	<code>npm audit --omit=dev</code> : 0 vulnerabilidades en producción; <code>npm audit</code> completo: 29 (14 altas, 6 moderadas, 9 bajas)

		webpack-dev-server, svgo). No llegan al navegador del usuario final.		
Media	Consola admin — Reportes	El detalle de accesos e impresiones se limitaba a 5.000 filas por consulta sin avisar; en un período con mucho tráfico, el total mostrado era el de las filas descargadas, no el real.	Corregido en este ciclo	Totales ahora vienen de un recuento exacto (<code>countRows</code>); si hay muestreo, se avisa en pantalla. <code>tests/admin/reports.spec.ts</code>
Media	Consola admin — Reportes (gráfico 3D)	Si el navegador no soportaba WebGL, la librería <code>echarts-gl</code> lanzaba una excepción no controlada y todo el reporte fallaba, incluida la tabla de datos.	Corregido en este ciclo	El fallo ahora se aisló al gráfico; la tabla con los mismos datos sigue mostrándose. Prueba: "si el gráfico 3D no carga se avisa y la tabla sigue mostrando los datos"
Baja	Centro de Seguridad	No se verificó en esta revisión que las alertas configuradas (webhook, correo) efectivamente se entreguen en producción; solo se auditó que el código las construye de forma segura (sin SSRF, con timeout).	Pendiente de verificación operativa	<code>check-security-alerts</code> (función Edge), migraciones <code>052</code> , <code>054</code> , <code>055</code>
Baja	Base de datos — gestión de roles	Cambiar el rol <code>admin</code> de un usuario vía SQL Editor requiere pasos no evidentes (<code>set_config</code> de <code>service_role</code>), porque el editor no se autentica como	Aceptado; falta documentar	Trigger <code>prevent_user_security_field_tampering</code> , <code>005_security_hardening.sql:21-23</code>

		ningún rol de aplicación. No es una debilidad — es la protección funcionando — pero no está documentado para quien administre la base.		
Baja	Documentación	No existe un documento único de modelado de amenazas; las decisiones de seguridad (por qué SSRF se bloquea así, por qué el sorteo es atómico) viven repartidas en comentarios de código y migraciones.	Pendiente	—

Cambios y mejoras necesarios

Orden	Acción	Esfuerzo estimado	Justificación
1	Verificar en producción que una alerta real del Centro de Seguridad llega (disparar una de prueba y confirmar recepción)	Horas	Es la única forma de saber si el sistema de alertas funciona de punta a punta; el código por sí solo no lo garantiza
2	Actualizar <code>react-scripts</code> o migrar a un toolchain de build mantenido (p. ej. Vite)	Días	Elimina los 14 avisos altos; no es urgente porque no afecta al usuario final, pero se acumula con el tiempo
3	Escribir una guía corta de "cómo cambiar el rol de un usuario" para quien administre la base de datos	Menos de un día	Evita que alguien intente desactivar el trigger de seguridad por no entender el error
4	Documento breve de modelado de amenazas (una página: qué protege cada control y qué pasa si falla)	1-2 días	Acelera la incorporación de nuevas personas al equipo y sirve de referencia ante una auditoría externa

	Idioma		Externa
5	Definir y documentar cuánto tiempo se conservan los registros del Centro de Seguridad	Horas	Hoy no hay una política escrita; importa si en algún momento se necesita investigar un incidente pasado
6 (opcional)	Función SQL agregada para el reporte de Reportes, sin el límite de 5.000 filas	Día	Solo mejora exactitud en volúmenes muy altos; los totales que se muestran hoy ya son exactos

Ninguno de estos seis puntos bloquea el uso actual del sistema; los tres primeros son los que más reducen riesgo por el esfuerzo que piden.

Límites de este análisis

- **Es estático, no dinámico.** No se intentó explotar nada contra el sistema en producción (sin fuzzing, sin intentos reales de inyección, sin escaneo de puertos).
- **No cubre la infraestructura de Google Cloud Run ni la configuración de red** (firewalls, IAM del proyecto, cifrado en tránsito más allá de HTTPS).
- **No incluye una revisión de terceros** (Supabase, Google Cloud, los proveedores de IA usados por la app) más allá de cómo este código los llama.
- **Las pruebas automatizadas de Playwright y Jest** (85 de Jest, cientos de Playwright) respaldan el comportamiento descrito, pero una prueba automatizada demuestra que el caso que cubre funciona, no que no exista un caso sin cubrir.
- Fecha de corte de esta revisión: commit `41569fc` , 2026-09-21. Cambios posteriores no están reflejados aquí.